

Spatial Mental Modeling from Limited Views

Dissertation Progress Report — Milestone 1

A Prompt-Specification Gap in ViperGPT and Its Consequences for Reproducibility

Student Sankarshan
Supervisor Dr. Shrikant Malviya
Programme M.Tech Data Science, SVNIT Surat
Date 11 August 2026

Summary. ViperGPT (Surís et al., ICCV 2023) answers visual queries by prompting a code-generating LLM to emit a Python program that calls vision modules. Its original generator, OpenAI Codex, has since been retired, so any present-day reproduction must substitute an open-weights model. We show that this substitution exposes an unstated assumption in the released prompt: it never specifies that a visual-grounding program must return an `ImagePatch`. With Qwen2.5-Coder-7B-Instruct as the generator, **98.8%** of generated programs return a natural-language string where a bounding box is required—syntactically valid, semantically fluent, and structurally incapable of scoring above zero. Adding an explicit return-type contract and two grounding exemplars reduces this to **0.2%**. We report the measurement, the fix, and its cost.

1 Background and Motivation

ViperGPT decomposes a visual query into an executable program rather than answering it end-to-end. A code-generating LLM π receives an API specification and a query, and emits a Python function `execute_command` that composes calls to pretrained vision modules. The claimed advantages are interpretability—the intermediate reasoning is inspectable code—and compositionality.

Two facts motivate this work:

1. **The original generator no longer exists.** ViperGPT used Codex (`code-davinci-002`), which OpenAI has retired. Every reproduction attempted today must substitute a different model. The paper does not discuss what this substitution costs.
2. **The spatial API is thin.** The framework exposes `compute_depth` (a single scalar: the median relative depth of a patch) and `distance` (pixel separation). There is no camera geometry, no metric scale, no 3D position. This is the gap our dissertation ultimately targets.

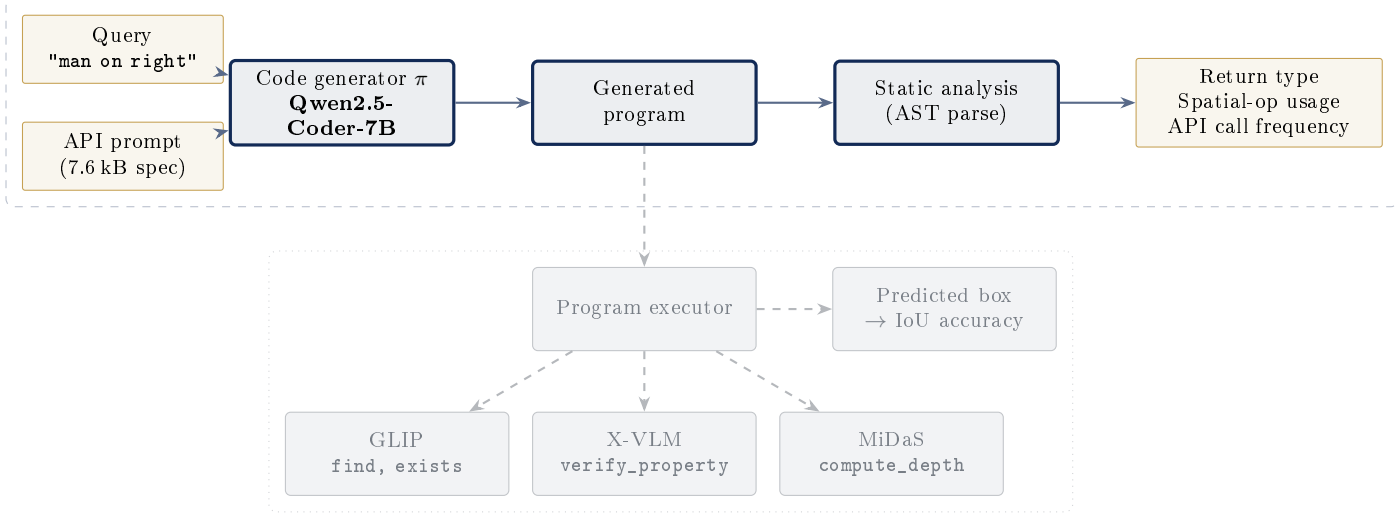
Milestone 1 addresses (1). It is deliberately sequenced first because it requires only the code generator—no perception modules, no images—and therefore yields a citable result independently of whether the perception stack builds successfully.

2 Method

2.1 Pipeline

Figure 1 shows the ViperGPT pipeline and the sub-path exercised in Milestone 1. We generate programs and analyse them *statically*: the programs are parsed, not executed. This isolates the code-generation stage from perception entirely.

Milestone 1 – executed (this report). No images, no perception modules.



Milestone 2 – not yet built. GLIP requires CUDA compilation; X-VLM wrapper is a stub.

Figure 1: The ViperGPT pipeline. Milestone 1 exercises the generation path only and analyses the emitted programs statically. The execution path is deferred.

2.2 Metrics

Because programs are not executed, we measure structural properties that *bound* downstream accuracy:

Parse rate. Fraction of generations that are syntactically valid Python.

Task-invalid return rate. Fraction whose `return` statement yields a string rather than an `ImagePatch`. Visual grounding is evaluated by IoU against a ground-truth box; a program returning a sentence cannot score, regardless of how good the perception modules are. This metric is therefore an *upper bound violation*: it identifies programs that are guaranteed to fail.

Spatial-constraint drop rate. Among queries containing a spatial relation, the fraction of programs containing no positional operation at all (`left`, `right`, `horizontal_center`, `compute_depth`, `distance`, ...)—i.e. the relation was silently discarded.

API call frequency. Which parts of the exposed API the generator actually uses.

2.3 Experimental setup

Component	Configuration
Generator π	Qwen2.5-Coder-7B-Instruct, bfloat16, greedy decoding
Max new tokens	320
Datasets	RefCOCO / testA, RefCOCO+ / testA (UNC split)
Queries per run	500
Hardware	1×NVIDIA H100 NVL (95 GB), SLURM <code>-gres=shard:25</code>
Runtime	≈63–73 minutes per full run (both datasets)
Jobs	29187 (baseline prompt), 29193 (grounding prompt)

RefCOCO permits spatial relations in its referring expressions; RefCOCO+ explicitly forbids them. This gives a built-in control: 57.8% of our RefCOCO sample contains a spatial relation, against 9.4% for RefCOCO+.

3 Finding: the released prompt omits the grounding contract

3.1 Diagnosis

The released `api.prompt` is task-agnostic—the same specification serves question answering and visual grounding. Two properties make it actively misleading for grounding:

1. Its in-context examples predominantly return strings: `return foo_patch.simple_query(...), return foo_patches[0].best_text_match(...), return building_patch.llm_query(...)`. Nothing states that a grounding program must return a patch.
2. It explicitly instructs: *“Use base Python (comparison, sorting) for basic logical operations, left/right/up/down, math, etc.”* The geometric API is thereby discouraged by design.

Point (2) explains an otherwise puzzling call distribution: against 547 uses of `find`, `compute_depth` appears 29 times and `distance` 5 times. The geometric API is essentially unused—not because the generator fails, but because the prompt tells it not to.

3.2 Effect on generated programs

Baseline prompt **invalid**

```
# Query: "man on right"
def execute_command(image) -> str:
    image_patch = ImagePatch(image)
    man_patches = image_patch.find("man")
    if man_patches:
        man_patch = max(man_patches,
                        key=lambda m: m.
                            right)
        return man_patch.simple_query(
            "Who is on the right?")
    else:
        return "No man found."
```

Returns a sentence. IoU undefined.

With grounding contract **valid**

```
# Query: "man on right"
def execute_command(image) -> ImagePatch:
    image_patch = ImagePatch(image)
    man_patches = image_patch.find("man")
    man_patches.sort(
        key=lambda p: p.horizontal_center
    )
    return man_patches[-1]
```

Returns a patch. Spatial relation resolved.

A second illustrative case: for the query `"left black shirt"`, the baseline program finds shirts, verifies the colour, and returns `"The image contains a black shirt."`—the word *left* never appears in the program at all. Under the revised prompt the same query yields `shirt_patches.sort(key=lambda p: p.left); return shirt_patches[0]`.

3.3 The intervention

Three sentences and two exemplars were appended to the prompt: a statement that the function must return a single `ImagePatch` and not a string, bool, or list; an instruction that every spatial relation must be resolved rather than dropped; and two worked grounding examples (one resolved by 2D sorting, one by depth comparison). No other change was made. The original file is preserved as `prompts/api.prompt.orig`.

4 Results

Reading the table. The task-invalid return rate falls by more than two orders of magnitude. The spatial-constraint drop rate falls roughly tenfold on both datasets. Both effects are large, consistent across two datasets with different spatial composition, and reproduced between a 30-query pilot (0.0%) and the full 500-query run (0.2%).

Table 1: Effect of the grounding contract. Identical 500-query samples, identical model and decoding; only the prompt differs.

Metric	Baseline	+ Contract	Direction
<i>RefCOCO / testA</i> ($n = 500, 289$ spatial)			
Task-invalid return rate	98.8%	0.2%	improved
Spatial constraint dropped	17.3%	1.7%	improved
Parse rate	99.8%	96.6%	degraded
Mean API calls per program	2.45	1.66	simpler
<i>RefCOCO+ / testA</i> ($n = 500, 47$ spatial)			
Task-invalid return rate	98.2%	0.0%	improved
Spatial constraint dropped	31.9%	2.1%	improved
Parse rate	99.6%	92.6%	degraded
Mean API calls per program	2.89	2.27	simpler

Table 2: API call frequency under the baseline prompt, RefCOCO/testA, 500 queries. The geometric API is almost untouched.

Call	Count	Call	Count
find	547	overlaps_with	23
llm_query	265	crop	22
simple_query	207	best_text_match	16
verify_property	109	distance	5
compute_depth	29	exists	2

The cost is real and is reported. Parse rate falls by 3.2 points on RefCOCO and 7.0 points on RefCOCO+. The stricter contract induces a small but non-trivial rate of syntactically invalid generations. Net structural validity nonetheless improves from roughly 1% to roughly 96%.

A note on llm_query. It is invoked 265 times under the baseline prompt. It is a text-only model with no access to the image, yet is asked questions such as “*Who is in the bottom left corner of the image?*”. These calls cannot succeed even in principle.

5 Interpretation

The result is best read as a *reproducibility* finding rather than a criticism of ViperGPT’s method. Codex plausibly inferred the grounding contract from the task context without being told; the paper therefore had no reason to state it. But Codex is retired, so every present-day reproduction substitutes an open-weights model, and every such reproduction will encounter a near-total structural failure with no diagnostic pointing at the cause. The failure is silent: programs parse, execute without error, and return fluent English. Only IoU evaluation—which requires the full perception stack to be built first—would reveal that the score is zero.

The practical implication is that prompt specifications released alongside LLM-driven systems must state their output contracts explicitly, because the contract was previously carried implicitly by a model that is no longer available.

6 Reproducibility record

6.1 Documented deviations from the paper

ID	Deviation	Reason and expected effect
D1	Codex $\rightarrow$ Qwen2.5-Coder-7B-Instruct	Codex is retired. This substitution is the subject of the present study.
D3	RefCOCO annotations from Internet Archive	The canonical UNC host no longer resolves (verified from two networks). Archived zips are byte-identical. No effect on results.
D4	torch 2.13/CUDA 13, transformers 5.15 instead of pinned 2.1/4.36	The conda environment pre-existed, so pins were not applied. No effect on the text-only path; a material risk for Milestone 2.

6.2 Scope

Restricted to RefCOCO / RefCOCO+ (visual grounding). GQA, OK-VQA and NExT-QA are out of scope: the four-table configuration requires roughly 250 GB against 231 GB of available quota, and NExT-QA alone requires over 100 GB of video.

6.3 Artefacts

Every run writes a timestamped directory containing the resolved configuration, the per-query program dump (`programs.jsonl`), aggregate counts (`analysis.json`) and a rendered table (`analysis.md`). SLURM job IDs are recorded. A dated research log (`docs/research_log.md`) records every failed attempt, including the rejected initial hypothesis described below.

6.4 A rejected hypothesis, recorded

Milestone 1 was originally designed to test a different claim: that spatial-relational queries would cause the generator to collapse the entire query into a single opaque `simple_query` call, as the paper’s GQA prompt explicitly instructs. The measured collapse rate was $\approx 0\%$ everywhere. The instruction in question applies to GQA, not to grounding, and grounding cannot use `simple_query` because it must return a patch. The hypothesis was well-posed but aimed at the wrong task. The cost of establishing this was one 63-minute job, incurred before any perception model had been built—which is precisely why Milestone 1 was sequenced ahead of Milestone 2.

7 Limitations

1. **Static analysis only.** Programs are parsed, not executed. We can show that 98.8% of baseline programs are structurally incapable of scoring, but we cannot yet report IoU accuracy for either condition. That requires the perception stack.
2. **One generator, one seed.** All results use Qwen2.5-Coder-7B-Instruct with greedy decoding. Whether the effect is model-scale dependent is untested.
3. **Heuristic spatial classification.** Spatial queries are identified by a relational-term lexicon. Spot checks are correct ("man sitting on bench" $\rightarrow$ non-spatial; "person on left of bench" $\rightarrow$ spatial), but no human-annotated ground truth exists for this label.
4. **Return-type detection is syntactic.** A program returning a variable is counted as valid without proving that the variable holds a patch.

8 Next steps

1. **Generator sweep** (days). Repeat on Qwen2.5-Coder-1.5B and 32B to establish whether the effect is scale-dependent. Requires no new infrastructure.
2. **Third prompt variant** (days). Remove the instruction that discourages the geometric API and measure whether `compute_depth` and `distance` usage rises. This yields a three-point ablation: baseline $\rightarrow$ contract $\rightarrow$ geometric encouragement.
3. **Milestone 2: execution** (weeks). Build GLIP, implement the X-VLM wrapper, fetch COCO images, and report IoU accuracy for each prompt condition. The principal risk is that the GLIP fork’s custom CUDA kernels predate the `sm_90` architecture of the available H100s and may not compile against CUDA 13.
4. **Dissertation contribution**. Replace the scalar `compute_depth` with a metric 3D representation exposing `position_3d`, `distance_3d`, and `is_behind`, so that spatial relations become executable geometric operations rather than 2D pixel arithmetic. Milestone 1 establishes the baseline against which that contribution will be measured.

All figures in this report are traceable to a run directory, a configuration, and a SLURM job ID. Numbers reported here are from jobs 29187 and 29193, executed 10–11 August 2026.